Smart Virtual Assistant - Proposal Project

---

1. The Ill-Defined Problem

1.1 Problem Statement

Students who recently moved to the city or are unfamiliar with local routes often face difficulties in finding food vendors that match their budget and taste preferences. This challenge is especially prominent when they have not yet built a network with local friends or residents who can assist them in navigating the area.

1.2 Ambiguous & Edge Test Cases

This section outlines ambiguous user inputs, qualitative criteria, or edge cases that require clarification, threshold handling, or default fallback mechanisms from the system:

- **Test Case 1 (Irrelevant Input):**
    - *Input:* User inputs irrelevant or non-food-related information.
    - *Expected Output:* Prompt the user to re-enter input or display an error message.
- **Test Case 2 (Missing Location):**
    - *Input:* User does not provide their current location.
    - *Expected Output:* Prompt the user to request location access/input.
- **Test Case 3 (Food Safety & Quality Assessment):**
    - *Input:* A restaurant/vendor offers lower prices, but its quality and food safety standards are not guaranteed compared to other options.
    - *Expected Output:* Assign a low ranking/priority score to the vendor.

---

2. The Well-Defined Problem

This section outlines deterministic scenarios with clear inputs, structured rules, and explicit logic:

- **Test Case 1 (Budget-Oriented Search):**

    - *Input:* User provides only maximum budget/spending limit.
    - *Expected Output:* Return top-rated vendors sorted by lowest price (Prioritize Price).

- **Test Case 2 (Dish-Oriented Search):**

    - *Input:* User provides only the desired dish name.
    - *Expected Output:* Return vendors serving that dish, starting from the lowest price with high rating scores (Prioritize Rating).

- **Test Case 3 (Cost vs. Distance Optimization - Fuel Cost Exceeds Price Savings):**

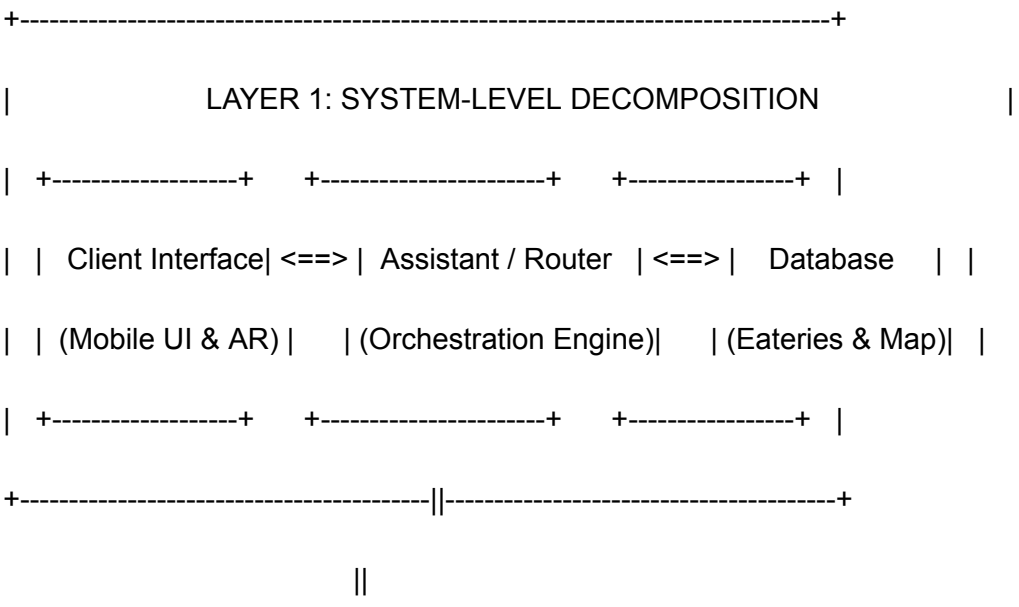
    - *Input:* A vendor is cheaper and higher rated, but located significantly further away such that the estimated fuel cost exceeds the price difference compared to closer options.
    - *Expected Output:* Recommend the closer vendor option.

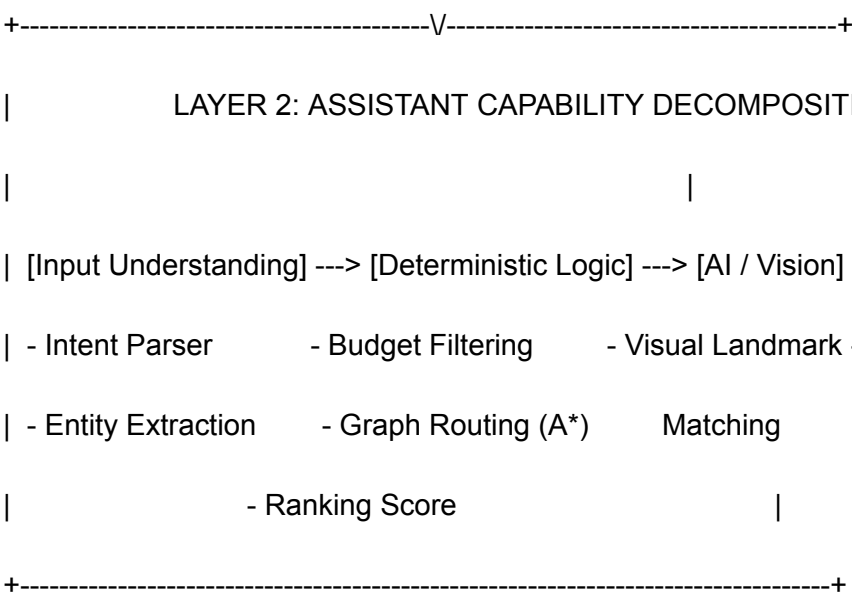
---

3. The Four Pillars of Computational Thinking

3.1 Decomposition

The system architecture is organized into a **2-Layered Decomposition Model**, cleanly isolating system orchestration from granular computational capabilities [cite: 6].





3.2 Pattern Recognition

Instead of treating user queries as dozens of isolated functions, HEANA categorizes all operations into **4 Core Computational Patterns** [cite: 6].

Core Computational Patterns

Pattern Group ID	**Pattern Name**	**Included Sub-problems**	**Recognition Criteria**	**Processing Mechanism**
PAT-01	**Query & Constraint Pattern**	Search eatery, Filter by conditions, Search by radius, Information lookup	User provides at least one clear parameter (dish, budget, radius, opening hours).	Extract entities $\rightarrow$ Convert to structured parameters `{dish, max_price, radius}` $\rightarrow$ SQL/NoSQL Database Filtering.

Pattern Group ID	Pattern Name	Included Sub-problems	Recognition Criteria	Processing Mechanism
PAT-02	**Ranking & Multi-Criteria Pattern**	Rank results, Optimal recommendations	User requests the "best", "cheapest", "closest", or trade-offs across parameters.	Apply Weighted Scoring: S(r) = w_P x P_r + w_Q x Q_r + w_D x D_r + w_R x R_r (this formula will be described in algorithm design part).
PAT-03	**Ambiguous & Clarification Pattern**	Clarify requests, Missing input handling	Essential context missing (e.g., missing location/budget) or out-of-scope input.	Trigger Clarification Prompt or inject default safe fallback context.
PAT-04	**Null Result & Edge Handling Pattern**	Handle empty results, Safety/Hygiene alerts	Constraints yield zero matches or candidates fail health/safety thresholds.	Constraint Relaxation (expand radius/price) or attach warning tags.

3.3 Abstraction

Through **Abstraction**, real-world complexities—such as staff attitude, weather noise, conversational slang, and environmental distractions—are systematically stripped away. The problem is reduced to computable mathematical functions and standardized data objects [cite: 6].

3.3.1 Data Abstraction Models

- **Vendor Profile Abstraction:** `VendorID`, `Coordinates (lat, lng)`, `PriceRange (min_price, max_price)`, `VerifiedStatus (Boolean)`, `RatingScore (0.0 - 5.0)` [cite: 6].
- **User Profile Abstraction:** `UserLocation (Coordinates)`, `MaxBudget (Integer)`, `Allergies (List<String>)`, `TargetDish (String)` [cite: 6].

- **Alley Graph Network Abstraction:** The physical alley network is abstracted into a Weighted Graph $G = (V, E)$, where Vertices (V) represent intersections/landmarks, Edges (E) represent alley segments, and Weight (W) represents physical distance in meters [cite: 6].

3.3.2 Stripping Noise Across Tasks

Task	Core Mathematical Model	Stripped / Omitted Noise
Intent Parsing	$f_{\text{NLU}}(\text{Text Input}) \rightarrow {\text{Intent}, \text{Entities}}$	Filler words, emotional cravings ("ngon ngon"), slang ("35 cành" $\rightarrow 35000$).
Budget Checking	$S = {v \in \text{Database} \mid v.\text{Price} \le \text{MaxBudget} \land v.\text{Status} = \text{'OPEN'}}$	Vendor ownership history, seating capacity, plastic/wooden tables, air conditioning.
Graph Routing	Min Shortest Path $f(n) = g(n) + h(n)$ on $G(V, E)$	Power lines, wall paint/graffiti, weather conditions, roadside trash, ambient noise.
Ranking Score	$\text{Score} = (0.5 \cdot \text{Rating}) - (0.3 \cdot \text{Distance}) - (0.2 \cdot \text{Price})$	Long anecdotal text reviews, subjective complaints about waiting time, visual "charm".
Landmark Matching	$\text{MatchScore} = f_{\text{match}}(I_{\text{cam}}, I_{\text{ref}}) \ge \theta_{\text{threshold}}$	Moving pedestrians, motorbikes driving past, cloud shadows, daytime/nighttime lighting glare.
Fallback System	State Machine Exception Handling	Continuous processing of dark/blurry frames, complex 3D AR rendering overheads.

3.4 Algorithmic Design

HEANA employs a sequential, multi-stage pipeline combining hard constraint filtering, real-world cost optimization, and weighted ranking [cite: 6].

Multi-Stage Execution Pipeline

1. **Input Verification & Validation Stage:**
    - Verifies data integrity. Aborts early and requests missing mandatory parameters (e.g., GPS coordinates) if inputs are invalid or out-of-scope [cite: 6].
2. **Candidate Retrieval Stage:**
    - Queries relational/spatial databases to retrieve candidate eateries matching target dishes and general search areas [cite: 6].
3. **Hard Filtering Stage:**
    - Removes candidates failing prerequisite conditions [cite: 6]:
        - **Budget:** Drops venues where $\text{Price} > \text{MaxBudget}$ [cite: 6].
        - **Safety & Allergens:** Drops unverified establishments or dishes containing ingredients present in the user's allergy list [cite: 6].
4. **Total Cost Model Stage:**
    - Calculates total real-world economic cost to prevent recommending cheap meals requiring expensive travel [cite: 6]: $$C_{\text{total}} = C_{\text{food}} + C_{\text{travel}}$$
    - C_{travel} is derived from real geographic distance combined with student transit mode [cite: 6].
5. **Multi-Factor Scoring & Ranking Stage:**
    - Computes composite ranking score $S(r)$ for surviving candidates [cite: 6]: $$S(r) = w_P \cdot P_r + w_Q \cdot Q_r + w_D \cdot D_r + w_R \cdot R_r$$
    - Weights balance price optimization (P_r), dish relevance (Q_r), distance/travel time (D_r), and rating (R_r) [cite: 6].
6. **Tie-Breaking Mechanism Stage:**
    - If composite scores $S(r)$ are identical, the system prioritizes the venue with the shortest physical distance to the user [cite: 6].

---

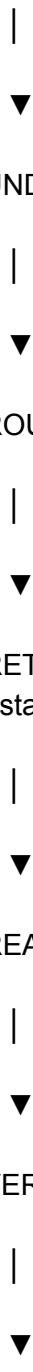
4. Solutions and Exploration Directions

4.1 System Interaction Pipeline (7-Step Architecture)

[User Request]



1. REQUEST —— Receives text/image via Web/Mobile Chatbot UI (HTTPS/WebSocket)



2. UNDERSTAND —— Parses intent, extracts entities (Location, Budget, Group Size, Tags)

3. ROUTE —— Directs payload to specialized modules (Budget Engine, Navigation, QA)

4. RETRIEVE/COMPUTE—— Queries Vector/Relational DB; calculates max per-person budget & distance

5. REASON —— LLM/Reasoning Engine synthesizes evidence, selects optimal option

6. VERIFY —— Guardrails check constraints (Budget <= Max, Open hours, Safety checks)

7. RESPOND/ACT —— Renders formatted cards/maps on Chat UI; logs operations

---

4.2 Case Study Execution Walkthrough

Scenario Setup

- **User Context:** A group of 3 HCMUS students at Campus 2 (Linh Trung - Di An) are searching for a quiet milk tea spot to study on a weekday afternoon, with a total combined budget of 100,000 VNĐ.

Step-by-Step Pipeline Execution

1. REQUEST

- **User Query:** *"There are 3 of us with a total budget of 100k. We want to find a quiet milk tea spot to study near Campus 2. Please give us easy step-by-step alley directions as well!"*

2. UNDERSTAND

NLU Parser extracts structured JSON:

```
{
  "intent": "Search_Hidden_Cafe_With_Navigation",
  "constraints": {
    "location": "CS2_LinhTrung",
    "people_count": 3,
    "total_budget": 100000,
    "purpose": "study_quiet",
    "item_type": "milk_tea",
    "requires_alley_navigation": true
  }
}
```

3. ROUTE

Router dispatches request in parallel to:

- **Budget Optimization Engine** (Item/Price combination matching)
- **Micro-Navigation Retrieval Module** (Alley landmark sequence extraction)

4. RETRIEVE / COMPUTE

- **Calculation:** Budget Per Person= 100,000\3 = 33,333 VNĐ/person
- **Query:** Search candidates where `location == CS2_LinhTrung`, `tags CONTAINS ["quiet", "power_outlets"]`, and `item_price <= 33000`.
- **Retrieved Candidate: "Tra Tro Cafe & Milk Tea"**
  - *Beverage Price:* 25,000 - 30,000VNĐ per item.
  - *Parking Fee:* 0VNĐ (Free student parking).
  - *Landmarks:* `[Dorm Gate A] -> [Alley 12] -> [Yellow Utility Pole] -> [Cat Cat Laundry] -> [Destination]`.

5. REASON

- Selected 3 standard milk teas at 28,000VNĐ each: Total Cost = 3 x 28,000VNĐ = 84,000 VNĐ <= 100,000 VNĐ
- Formatted detailed landmark navigation instructions.

6. VERIFY

- ✅ **Operating Hours Check:** Open (08:00 - 22:00), current time is 14:30 => **PASS**.
- ✅ **Budget Constraint Check:** 84,000VNĐ <= 100,000VNĐ => **PASS**.
- ✅ **Navigation Integrity Check:** Complete landmark sequence confirmed =>**PASS**.

7. RESPOND / ACT

The system returns the formatted output card to the student UI:

> **HCMUS Virtual Assistant Response:**
>
> ☕ **Recommended Venue: Tra Tro - Cafe & Milk Tea**
>
> - **Environment:** Quiet, spacious tables, equipped with power outlets for laptops.
>
> 💰 **Estimated Cost Breakdown:**
>
> - Traditional Milk Tea: $28,000\text{ VNĐ} \times 3 = 84,000\text{ VNĐ}$
> - Parking Fee: Free (0 VNĐ)
> - *Remaining Savings:* $16,000\text{ VNĐ}$ (out of 100,000 VNĐ budget)
>
> 🗺️ **Alley Navigation Steps:**

1. From **Dorm Gate A (KTX ĐHQG)**, walk straight for 100m.
2. **Turn right** into Alley 12 (opposite Com Tam eatery).
3. Walk 30m to the **yellow utility pole**, then turn left at **Cat Cat Laundry**.
4. Destination is at the end of the lane on your left (look for the wooden sign "Tra Tro").